



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ N 5

Название: «Шифрование файлов с открытым ключом (RSA + AES)»

Дисциплина: «Защита информации»

Студент ИУ7-72Б
(Группа)

(Подпись, дата)

Ю.А. Попов
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Ю.С. Руденкова
(И. О. Фамилия)

Москва, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Теоретические сведения	4
2 Реализация	5
2.1 Формат контейнера	5
2.2 Команды программы	5
3 Тестирование	6
4 Контрольные вопросы	7
ЗАКЛЮЧЕНИЕ	9
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	10
ПРИЛОЖЕНИЕ А. Основные листинги программы	11

ВВЕДЕНИЕ

Цель работы — реализовать потоковое шифрование произвольного файла по гибридной схеме RSA + AES. В работе реализованы RSAES-OAEP-SHA-256, AES-256-GCM, ручной AES-128 и консольные команды `keygen`, `encrypt`, `decrypt`.

1 Теоретические сведения

RSA использует пару ключей: открытый ключ передаётся отправителю, закрытый остаётся у получателя. Стойкость RSA связывают со сложностью факторизации модуля [1]. RSA медленен и ограничен размером сообщения, поэтому им защищается только сеансовый ключ.

AES — симметричный блочный шифр с блоком 128 бит [2]. Раунд AES включает SubBytes, ShiftRows, MixColumns и AddRoundKey. В AES-128, AES-192 и AES-256 выполняется соответственно 10, 12 и 14 раундов.

В программе файл шифруется AES-256-GCM, а ключ AES — RSAES-OAEP. GCM объединяет шифрование CTR и аутентификацию GHASH [3]. Любое изменение контейнера обнаруживается при сравнении 16-байтного тега. OAEP добавляет случайность RSA-преобразованию [4].

2 Реализация

2.1 Формат контейнера

Программа обрабатывает байты, поэтому тип файла не важен. Формат выходного файла приведён в таблице 2.1. Для RSA-2048 $L = 256$.

Таблица 2.1 — Формат контейнера HYB1

Смещение	Размер, байт	Поле
0	4	Сигнатура HYB1.
4	2	Длина L зашифрованного RSA-OAEP сеансового ключа.
6	L	Зашифрованный сеансовый ключ.
$6 + L$	12	Nonce AES-GCM.
$18 + L$	N	Шифртекст.
$18 + L + N$	16	Тег GCM.

Сеансовый ключ и nonce формируются `secrets.token_bytes`. Файл читается блоками 64 КиБ. При расшифровании данные сначала пишутся во временный файл; основной файл создаётся только после проверки GCM-тега.

2.2 Команды программы

```
1 python3 src/main.py keygen --public public.pem --private private.pem
2 python3 src/main.py encrypt --key public.pem --in source.rar --out source.enc
3 python3 src/main.py decrypt --key private.pem --in source.enc --out restored.rar
4 shasum -a 256 source.rar restored.rar
```

Листинг 2.1 — Команды консольного интерфейса

3 Тестирование

Тесты запускаются командой `python3 -m unittest discover -s tests -v`. Результаты приведены в таблице 3.1.

Таблица 3.1 — Результаты тестирования

N	Проверка	Результат
1	FIPS 197 C.1 для AES-128 и обратное преобразование.	Успешно.
2	NIST-вектор AES-256-GCM.	Успешно.
3	Верный и неверный пароль закрытого ключа.	Успешно.
4	Файл более двух буферов и неполный последний блок.	SHA-256 совпадает.
5	Повреждение шифртекста и чужой ключ.	Выходной файл не создан.

Все шесть автоматических тестов завершились успешно. Дополнительно выполнен полный цикл с RSA-2048 и двоичным файлом 70 КиБ. Отсутствие вывода стр подтверждает побайтное совпадение файлов.

```
• уроров2005@MacBook-Air-Urij-2 lab_05 % mkdir -p demo
• уроров2005@MacBook-Air-Urij-2 lab_05 % cp "Лабораторная_работа №5.docx" demo/source.docx
• уроров2005@MacBook-Air-Urij-2 lab_05 % python3 src/main.py keygen --public demo/public.pem --private demo/private.pem

Пароль закрытого ключа:
Повторите пароль:
Ключи сохранены за 9.02 с.
• уроров2005@MacBook-Air-Urij-2 lab_05 % python3 src/main.py encrypt --key demo/public.pem --in demo/source.docx --out demo/source.enc

Файл зашифрован: HYB1 / RSA-OAEP / AES-256-GCM.
• уроров2005@MacBook-Air-Urij-2 lab_05 % python3 src/main.py decrypt --key demo/private.pem --in demo/source.enc --out demo/restored.docx

Пароль закрытого ключа:
Файл расшифрован; SHA-256: 9c9b628ab3ff9f6caa7a1d736482ce9c169eef67c968bec3a0429978d27dccfc
```

Рисунок 3.1 — Вывод консоли при проверке программы

4 Контрольные вопросы

- 1) **Симметричные и асимметричные системы.** В первой используется общий секретный ключ, во второй — открытый и закрытый ключи.
- 2) **Почему AES не является алгоритмом с открытым ключом?** Для шифрования и расшифрования применяется один секретный ключ.
- 3) **Зачем совмещать RSA и AES?** RSA защищает ключ AES, AES быстро шифрует файл.
- 4) **Почему не шифруют файл RSA?** RSA медленен и ограничен размером модуля.
- 5) **Структура раунда AES.** SubBytes, ShiftRows, MixColumns, AddRoundKey.
- 6) **Нелинейность и рассеивание.** SubBytes обеспечивает нелинейность; ShiftRows и MixColumns — рассеивание.
- 7) **Число раундов AES.** 10, 12 и 14 для ключей 128, 192 и 256 бит.
- 8) **Почему ECB непригоден?** Одинаковые блоки дают одинаковый шифртекст и раскрывают структуру данных.
- 9) **Назначение IV и nonce.** Они исключают повторение шифртекста при одинаковых данных и ключе.
- 10) **Повтор nonce в GCM.** Повторяется поток CTR, что раскрывает связь открытых текстов и ослабляет аутентификацию.
- 11) **GCM и CBC.** GCM даёт шифрование и тег целостности; CBC сам тег не создаёт.
- 12) **Аутентифицированное шифрование.** Оно скрывает данные и выявляет их изменение.
- 13) **Генерация и стойкость RSA.** Выбирают простые p , q , вычисляют n , e , d ; стойкость основана на сложности факторизации n .
- 14) **Назначение OAEP.** OAEP добавляет случайность и защищает RSA от известных атак на учебную схему.
- 15) **Почему текст и RAR обрабатываются одинаково?** Оба файла являются последовательностями байтов.

- 16) **Хранение закрытого ключа.** Только у владельца в защищённом паролем PEM-файле; пароль защищает файл ключа.
- 17) **Неустранённые угрозы.** Схема не подтверждает отправителя и не защищает устройство; подлинность добавляет электронная подпись и сертификат.

ЗАКЛЮЧЕНИЕ

Реализована гибридная система RSAES-OAEP-SHA-256 + AES-256-GCM с форматом HYB1 и потоковой обработкой файлов. Реализован и проверен AES-128. Тесты подтвердили восстановление исходных данных, проверку SHA-256 и корректную обработку неверного пароля, ключа и повреждённого контейнера.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Rivest R. L., Shamir A., Adleman L. A Method for Obtaining Digital Signatures and Public-Key Cryptosystems // Communications of the ACM. — 1978. — Т. 21, № 2. — С. 120–126.
2. Technology. FIPS 197: Advanced Encryption Standard (AES) [Электронный ресурс]. — 2023. — Режим доступа: <https://csrc.nist.gov/pubs/fips/197/final>.
3. Technology. SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC [Электронный ресурс]. — 2007. — Режим доступа: <https://csrc.nist.gov/pubs/sp/800/38/d/final>.
4. Moriarty K.; Kaliski B.; Jonsson J.; Rusch A. PKCS N 1: RSA Cryptography Specifications Version 2.2 [Электронный ресурс]. — 2016. — Режим доступа: <https://www.rfc-editor.org/rfc/rfc8017.html>.

ПРИЛОЖЕНИЕ А. Основные листинги программы

```
def encrypt_file(source: str | Path, destination: str | Path, key: PublicKey) -> None:
    """Запись HYB1-контейнера при чтении файла фиксированными блоками."""
    session_key, nonce = secrets.token_bytes(32), secrets.token_bytes(12)
    wrapped = rsa_oaep_encrypt(session_key, key)
    keys, subkey, counter = _gcm_context(session_key, nonce)
    ghash, total = _GHash(subkey), 0
    output = _temporary(destination)
    name = output.name
    try:
        with Path(source).open("rb") as input_file, output:
            output.write(MAGIC + len(wrapped).to_bytes(2, "big") + wrapped + nonce)
            while chunk := input_file.read(BUFFER_SIZE):
                ciphertext, counter = _gcm_transform(chunk, keys, counter)
                output.write(ciphertext)
                ghash.update(ciphertext)
                total += len(ciphertext)
            output.write(_xor(_aes_encrypt_block(nonce + b"\0\0\0\1", keys),
ghash.finish(total)))
        os.replace(name, destination)
    except Exception:
        try:
            os.unlink(name)
        except FileNotFoundError:
            pass
        raise

def decrypt_file(source: str | Path, destination: str | Path, key: PrivateKey) -> None:
    """Публикует результат только после успешной проверки тега GCM."""
    source_path = Path(source)
    with source_path.open("rb") as input_file:
        size = source_path.stat().st_size
        header = input_file.read(6)
        if len(header) != 6 or header[:4] != MAGIC:
            raise ValueError("invalid encrypted-file signature (expected HYB1)")
        wrapped_size = int.from_bytes(header[4:], "big")
        if wrapped_size != key.size_bytes:
            raise ValueError("encrypted file belongs to a different RSA key size")
        wrapped, nonce = input_file.read(wrapped_size), input_file.read(12)
        ciphertext_size = size - 6 - wrapped_size - 12 - 16
        session_key = rsa_oaep_decrypt(wrapped, key)
        keys, subkey, counter = _gcm_context(session_key, nonce)
        ghash, processed = _GHash(subkey), 0
        output = _temporary(destination)
```

```

name = output.name
try:
    with output:
        while processed < ciphertext_size:
            chunk = input_file.read(min(BUFFER_SIZE, ciphertext_size - processed))
            plaintext, counter = _gcm_transform(chunk, keys, counter)
            output.write(plaintext)
            ghash.update(chunk)
            processed += len(chunk)
        tag = input_file.read(16)
        expected = _xor(_aes_encrypt_block(nonce + b"\0\0\0\1", keys),
ghash.finish(processed))
        if not hmac.compare_digest(tag, expected):
            raise ValueError("authentication failed: wrong key or damaged file")
        os.replace(name, destination)
except Exception:
    try:
        os.unlink(name)
    except FileNotFoundError:
        pass
    raise

```

Листинг 4.1 — Потокое шифрование и расшифрование файла

```

def main() -> int:
    arguments = build_parser().parse_args()
    try:
        if arguments.command == "keygen":
            if arguments.bits < 2048 or arguments.bits % 2:
                raise ValueError("RSA-модуль должен быть чётным и не менее 2048 бит")
            public, private = generate_key_pair(arguments.bits)
            save_public_key(arguments.public, public)
            save_private_key(arguments.private, private, _password(arguments.password,
confirm=True))
            print("Ключи сохранены.")
        elif arguments.command == "encrypt":
            encrypt_file(arguments.source, arguments.destination, load_public_key(arguments.key))
            print("Файл зашифрован: HYB1 / RSA-OAEP / AES-256-GCM.")
        else:
            private = load_private_key(arguments.key, _password(arguments.password))
            decrypt_file(arguments.source, arguments.destination, private)
            print(f"Файл расшифрован; SHA-256: {sha256_file(arguments.destination)}")
        return 0
    except (OSError, ValueError) as error:
        print(f"Ошибка: {error}", file=sys.stderr)
    return 2

```

Листинг 4.2 — Основная функция консольного интерфейса